AWS - REDHAT ENTERPRISE LINUX

AWS Practical Documentation

Hosting a Static Website on RHEL EC2 Using NGINX

Cloud Provider:	Amazon Web Services (AWS)
Service:	Amazon EC2
Operating System:	Red Hat Enterprise Linux 10.2
Web Server:	NGINX
Region:	Asia Pacific (Mumbai), ap-south-1
Instance Type:	t3.micro

1. Introduction

This practical demonstrates how to deploy a static website on an Amazon EC2 virtual machine running Red Hat Enterprise Linux (RHEL). NGINX is installed and configured as the web server, and the website is published through its document root directory.

The experiment begins with a simple HTML page branded for HAWKEYE – The Private Cloud. The deployment is then extended by downloading and extracting the Start Bootstrap Landing Page template and publishing its production-ready files through NGINX.

The website is accessed from a client computer using the EC2 instance's public DNS over HTTP.

2. Objective

- Launch an EC2 instance using Red Hat Enterprise Linux.
- Connect to the instance securely through SSH.
- Install NGINX and configure it to run as a system service.
- Create and host a custom HTML page.
- Download and extract a static website template using command-line utilities.
- Publish the template's static files through NGINX.
- Access the deployed website from a browser using the instance's public DNS.
- Understand Linux file permissions and SELinux file-context restoration.

3. Architecture and Topology

Figure 1 illustrates the logical architecture used in this experiment.

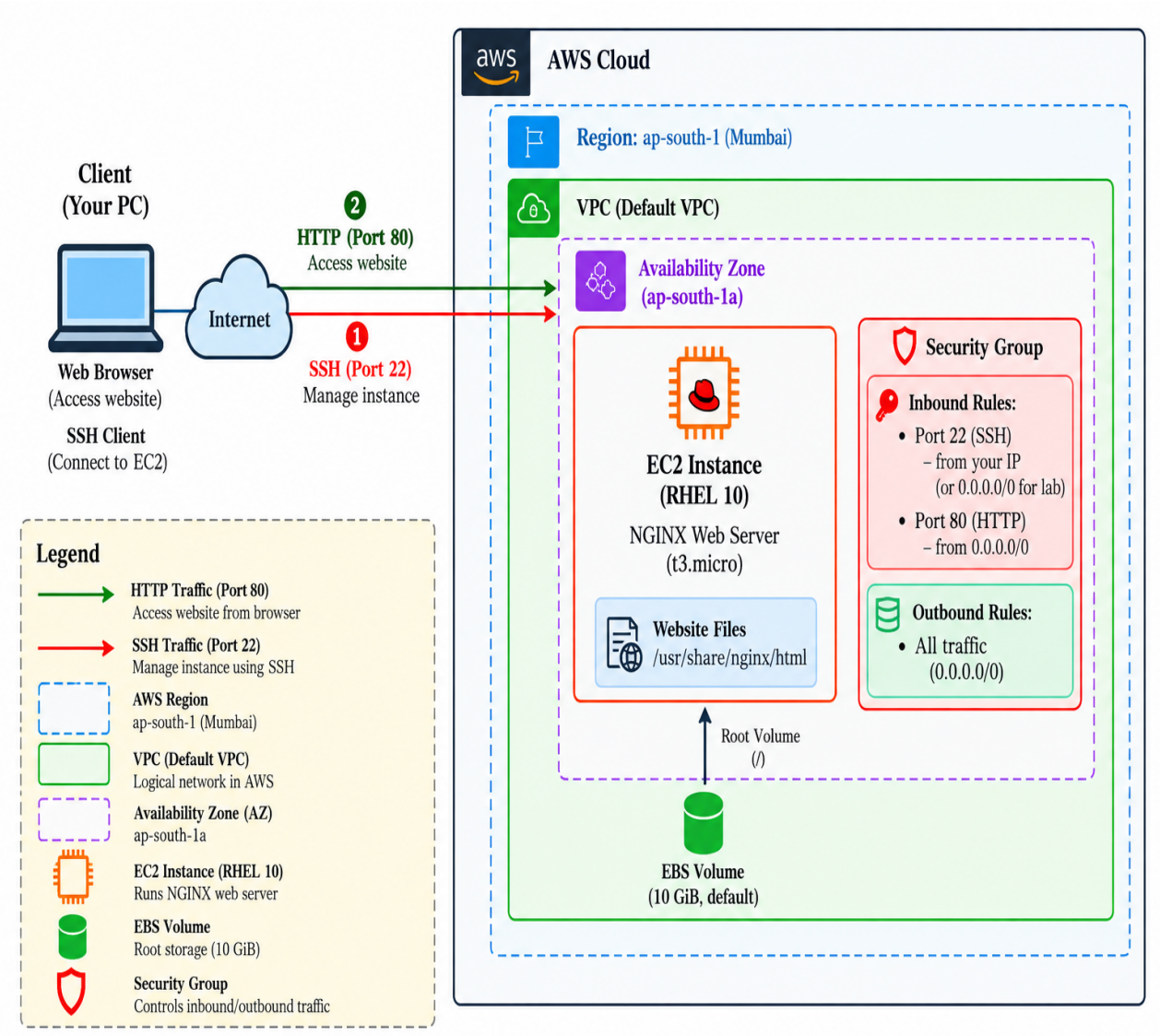


Figure 1: Topology for hosting a static website on EC2 using NGINX.

3.1 Components

- **Client computer:** Used to connect to the instance through SSH and view the website in a browser.
- **AWS Region:** The instance was launched in the Mumbai region, ap-south-1.
- **Amazon EC2:** Provides the virtual machine on which RHEL and NGINX run.
- **RHEL 10.2:** The operating system used for the practical.
- **NGINX:** Serves the HTML, CSS, JavaScript, and image files to HTTP clients.
- **Security group:** Controls network traffic reaching the instance. SSH uses TCP port 22, while HTTP uses TCP port 80.
- **EBS root volume:** Persistent block storage attached to the EC2 instance. The launch configuration showed a 10 GiB volume.
- **Web root:** The website files are published under `/usr/share/nginx/html`.

3.2 Traffic Flow

1. The administrator connects from the client computer to the EC2 instance using SSH on TCP port 22.
2. The administrator installs NGINX and copies the website files into the NGINX document root.
3. A browser sends an HTTP request to the instance's public DNS on TCP port 80.
4. NGINX returns the requested website files to the browser.

4. Prerequisites

The following were required for the practical:

- An AWS account with permission to launch EC2 instances.
- An EC2 key pair and its private `.pem` file.
- A security group allowing SSH on TCP port 22 from the administrator's IP address.
- An inbound HTTP rule allowing TCP port 80.
- A client computer with an SSH client and a web browser.

For security, SSH should normally be restricted to a trusted IP address rather than exposed to the entire Internet.

5. Implementation Steps

5.1 Launch the RHEL EC2 Instance

The instance was configured using the AWS EC2 launch wizard.

- **AMI:** Red Hat Enterprise Linux 10.
- **Instance type:** t3.micro.
- **Region:** ap-south-1 (Mumbai).
- **Key pair:** my-K-key.
- **Storage:** 10 GiB root volume.

After launching the instance, its state was checked until it became *Running*.

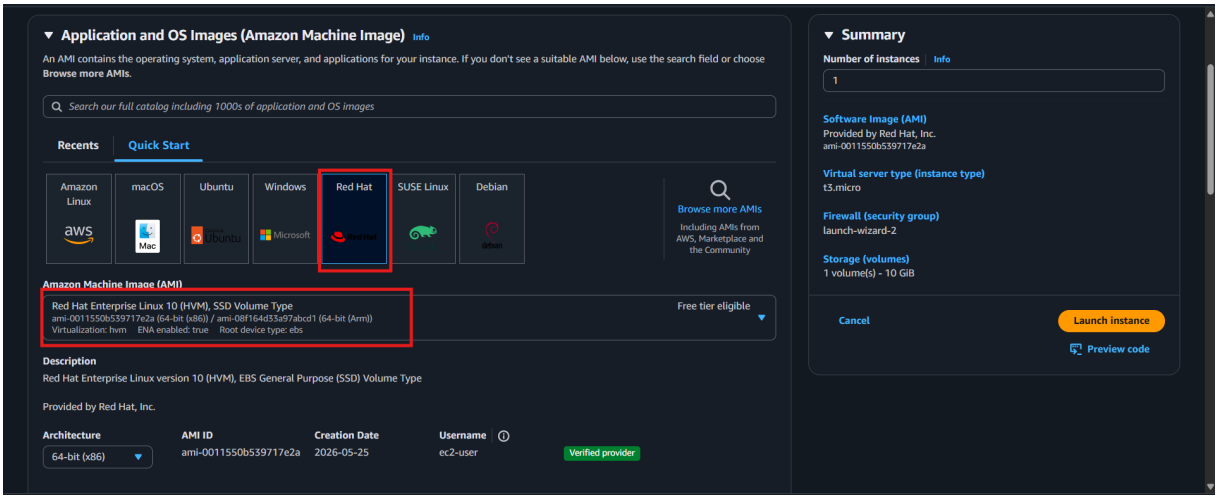


Figure 2: Selecting the Red Hat Enterprise Linux AMI.

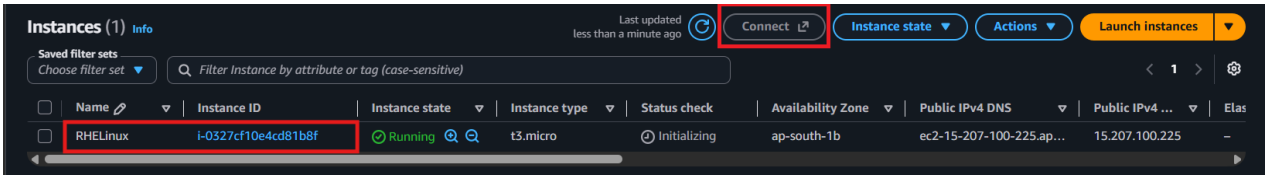


Figure 3: The EC2 instance in the Running state.

5.2 Connect to the Instance Using SSH

The SSH connection was initiated from the local Windows computer. The terminal was opened in the directory containing the downloaded private key.

```
cd Downloads
ssh -i "my-K-key.pem" ec2-user@ec2-15-207-100-225.ap-south-1.compute.amazonaws.
    com
```

Command breakdown:

- `cd Downloads`: Changes the current directory to the Downloads folder.
- `ssh`: Starts the Secure Shell client.
- `-i "my-K-key.pem"`: Specifies the private key used to authenticate the connection.
- `ec2-user@...`: Connects as the `ec2-user` account to the instance's public DNS.

The initial connection attempt timed out because SSH access was not yet authorized by the security group. After inbound TCP port 22 was allowed, the connection succeeded. On the first successful connection, the SSH client also asked whether to trust the host's fingerprint.

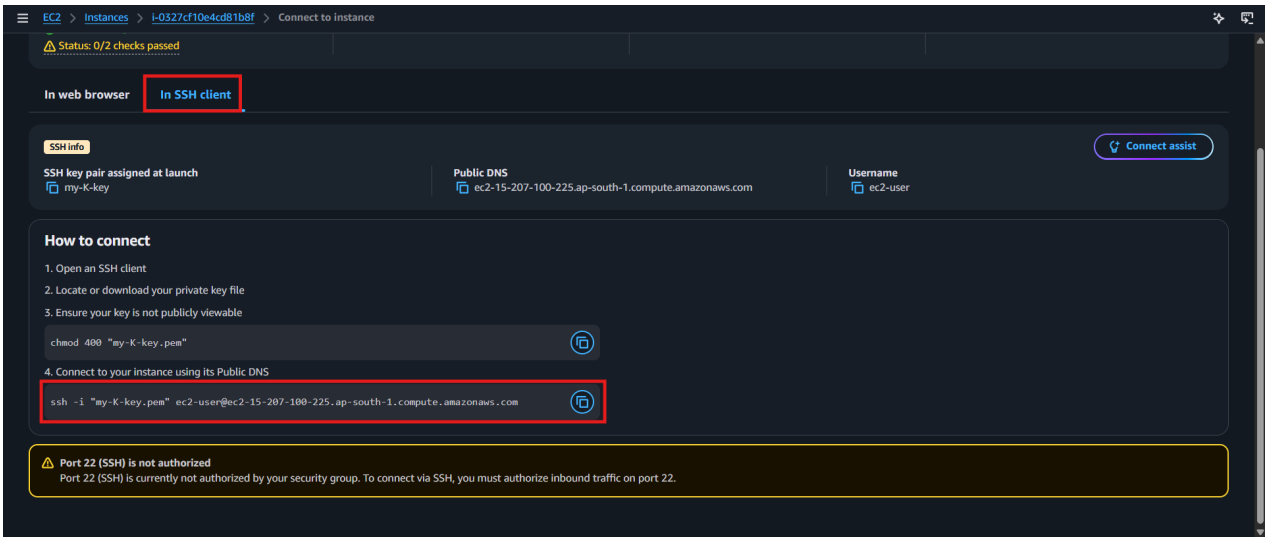


Figure 4: SSH connection instructions and the initial port 22 warning in the AWS console.

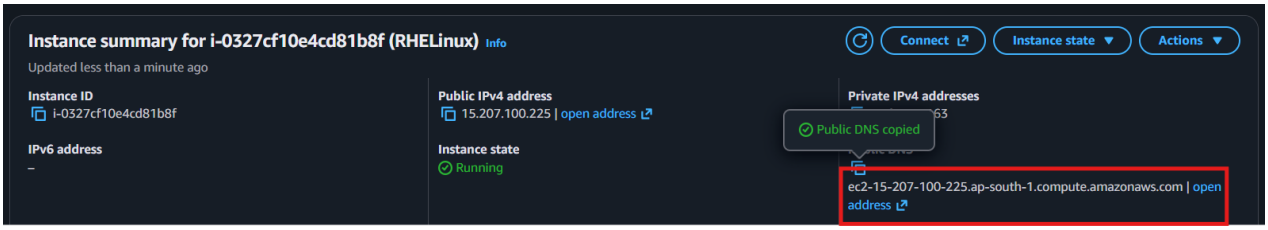


Figure 5: EC2 instance public IPv4 address and public DNS.

5.3 Verify the Operating System

After connecting, the following commands were used to identify the user, working directory, operating system, and package manager.

```
whoami
pwd
cat /etc/redhat-release
sudo dnf --version
```

Command breakdown:

- `whoami`: Displays the current username. The session used `ec2-user`.
- `pwd`: Prints the current working directory. The initial directory was `/home/ec2-user`.

- `cat /etc/redhat-release`: Displays the installed Red Hat Enterprise Linux release.
- `sudo dnf --version`: Displays the version of DNF, the package manager used by RHEL.

The operating system reported was Red Hat Enterprise Linux release 10.2.

5.4 Install and Start NGINX

NGINX was installed through DNF.

```
sudo dnf install nginx -y
```

Command breakdown:

- `sudo`: Executes the command with elevated privileges.
- `dnf`: Invokes the RHEL package manager.
- `install nginx`: Installs NGINX and its required dependencies.
- `-y`: Automatically answers yes to package installation confirmation prompts.

The service was then enabled and started.

```
sudo systemctl enable --now nginx
sudo systemctl status nginx --no-pager
```

Command breakdown:

- `systemctl`: Manages services controlled by systemd.
- `enable`: Configures NGINX to start automatically when the system boots.
- `--now`: Starts the service immediately in addition to enabling it.
- `status nginx`: Displays the current state of the NGINX service.
- `--no-pager`: Prints the status directly instead of opening an interactive pager.

The status output confirmed that NGINX was `active (running)`.

5.5 Understand the NGINX Web Root

The default web root used in this experiment was:

```
/usr/share/nginx/html
```

Files placed in this directory can be served by NGINX according to its configuration and file permissions.

The directory was inspected using commands such as `cd`, `ls`, and `ls -la`.

```
cd /usr/share/nginx/html
ls -la
```

Command breakdown:

- `cd`: Changes the working directory. An absolute path begins at the filesystem root.

- `ls`: Lists files and directories.
- `ls -la`: Includes hidden entries and displays detailed information such as permissions, ownership, file sizes, and symbolic links.

The initial NGINX page and supporting files were owned by `root`. Consequently, the ordinary `ec2-user` account could not directly create or remove files there.

5.6 Create a Basic HTML Page

The first website was a simple HTML page containing the heading "Welcome to HAWKEYE!" and a short description.

The initial attempts to use `touch` and shell redirection demonstrated that directory write permissions matter. For example, the following command does not make the output redirection privileged:

```
sudo echo "Hello World" > index.html
```

The shell processes `>` before the command runs under `sudo`. Therefore, the redirection is still performed with the permissions of the current shell.

A working command used a privileged shell for the redirection:

```
echo '<!DOCTYPE html>
<html>
<head>
    <title>HAWKEYE Private Cloud</title>
</head>
<body>
    <h1>Welcome to HAWKEYE!</h1>
    <p>My first webpage hosted on RHEL using NGINX.</p>
</body>
</html>' | sudo sh -c 'cat > /usr/share/nginx/html/index.html'
```

Command breakdown:

- `echo`: Outputs the HTML text.
- `|`: Sends the output of one command to the standard input of the next command.
- `sudo sh -c`: Starts a shell with elevated privileges and executes the quoted command.
- `cat > path`: Reads the supplied input and writes it to the specified file, replacing its existing contents.

5.7 Alternative Method: Edit the Page Using nano (Recommended)

Instead of using `echo` and shell redirection, the page can be created and edited interactively with the `nano` text editor. This is often easier when writing or modifying HTML manually.

Step 1: Install nano if it is not already installed.

```
sudo dnf install nano -y
```

Step 2: Open the HTML file with elevated privileges.

```
sudo nano /usr/share/nginx/html/index.html
```

If the file does not exist, nano opens a new buffer and creates it when the contents are saved.

Step 3: Enter the HTML content.

```
<!DOCTYPE html>
<html>
<head>
    <title>HAWKEYE Private Cloud</title>
</head>
<body>
    <h1>Welcome to HAWKEYE!</h1>
    <p>My first webpage hosted on RHEL using NGINX.</p>
</body>
</html>
```

Step 4: Save and exit nano.

1. Press `Ctrl+O` to write the file.
2. Press `Enter` to confirm the filename.
3. Press `Ctrl+X` to exit the editor.

Using `sudo nano` allows the editor to write the root-owned web file without changing its permissions or making the directory broadly writable.

5.8 Install wget and unzip

To retrieve the website template directly from the terminal, the `wget` and `unzip` utilities were installed.

```
sudo dnf install wget unzip -y
```

Command breakdown:

- `wget`: Downloads files from HTTP and HTTPS URLs, among other supported protocols.
- `unzip`: Extracts files from ZIP archives.
- `-y`: Automatically confirms installation.

The version checks were useful for confirming installation, but are not required every time:

```
wget --version
unzip -v
```

5.9 Download the Static Website Template

The download and extraction were performed in a separate working directory rather than keeping the archive and source project in the public web root.

```
mkdir -p ~/website-project
cd ~/website-project
wget https://github.com/StartBootstrap/startbootstrap-landing-page/archive/refs/
    heads/master.zip -O website.zip
```

Command breakdown:

- `mkdir`: Creates a directory.
- `-p`: Creates missing parent directories and does not report an error if the directory already exists.
- `~`: Represents the current user's home directory.
- `cd`: Moves into the project directory.
- `wget URL`: Downloads the ZIP archive from the specified GitHub repository.
- `-O website.zip`: Saves the downloaded response to the local filename `website.zip`.

The archive was approximately 4.1 MiB. The download output confirmed that the request completed successfully.

5.10 Extract the Archive and Locate the Build Files

```
unzip website.zip
ls
find . -path '*/dist/index.html'
```

Command breakdown:

- `unzip website.zip`: Extracts the contents of the archive into the current directory.
- `ls`: Lists the extracted project directory and the downloaded ZIP archive.
- `find .`: Searches recursively from the current directory.
- `-path '*/dist/index.html'`: Filters the search results to paths ending in the production distribution's `index.html`.

The search returned:

```
./startbootstrap-landing-page-master/dist/index.html
```

The `dist` directory contained the ready-to-publish HTML, CSS, JavaScript, and image assets. The source files, scripts, and package metadata were not needed to serve the static site.

5.11 Publish the Website Files

The production files were copied into the NGINX web root.

```
sudo cp -a ~/website-project/startbootstrap-landing-page-master/dist/. /usr/
    share/nginx/html/
```

Command breakdown:

- `cp`: Copies files and directories.
- `-a`: Archive mode; preserves relevant file attributes while copying recursively.

- `dist/.`: Copies the contents of the `dist` directory, including its hidden entries, rather than creating an extra `dist` folder inside the web root.
- `sudo`: Provides the permissions needed to write to the root-owned NGINX directory.

The resulting web root contained `index.html`, `css/`, `js/`, and `assets/`.
The ZIP archive and complete source project remained in `~/website-project`, outside the public web directory.

5.12 Restore SELinux File Contexts

The following command was run after publishing the files:

```
sudo restorecon -Rv /usr/share/nginx/html
```

Command breakdown:

- `restorecon`: Restores file security contexts to the values defined by the SELinux policy.
- `-R`: Applies the operation recursively to files and subdirectories.
- `-v`: Displays the changes made.
- `/usr/share/nginx/html`: Specifies the directory containing the published website.

The command output showed files changing from `user_home_t` to `httpd_sys_content_t`. This restored the expected SELinux content label for the NGINX-served files.

6. Results and Verification

6.1 Initial HTML Page

The first browser test displayed the custom HAWKEYE heading and the message identifying the page as the first website hosted on RHEL using NGINX.

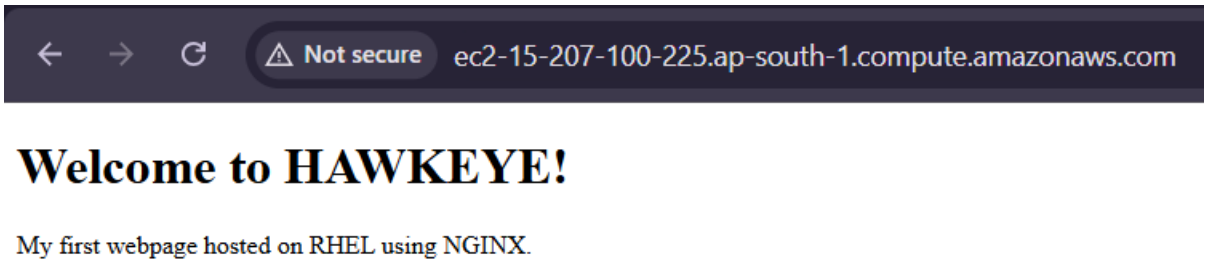


Figure 6: Initial HTML page served through NGINX.

6.2 Final Landing Page

After the Bootstrap distribution files were copied to the web root, the browser displayed the complete landing page, including its navigation, hero section, email form, and supporting sections.

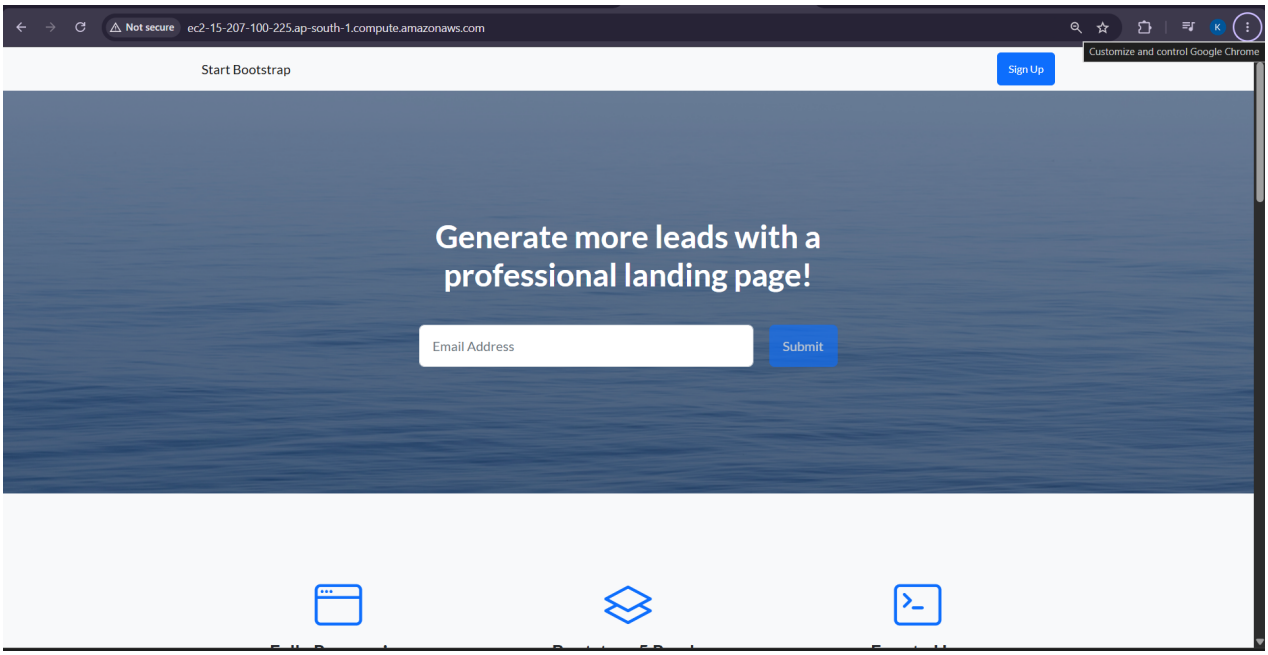


Figure 7: Final Start Bootstrap landing page hosted on EC2.

The website was accessed using the instance's public DNS:

```
http://ec2-15-207-100-225.ap-south-1.compute.amazonaws.com
```

The browser screenshot confirmed that the site was reachable over HTTP. The URL used HTTP rather than HTTPS, so the connection was not encrypted.

7. Important Concepts Learned

- **EC2:** Provides virtual compute capacity on which an operating system and applications can run.
- **SSH:** Enables secure remote terminal access to the Linux instance.
- **Security groups:** Control inbound and outbound network traffic for the instance.
- **NGINX:** Serves static web content to clients making HTTP requests.
- **Linux ownership and permissions:** Files owned by root cannot automatically be modified by an ordinary user.
- **Shell redirection:** Using `sudo` before `echo` does not make the shell's `>` redirection privileged. A privileged editor or shell is required.
- **Static deployment:** HTML, CSS, JavaScript, and image files can be served without running a backend application.
- **SELinux:** Security contexts help the operating system control how services access files.
- **Separation of source and published files:** Only the required distribution files should be placed in the public web root.

8. Conclusion

The practical successfully demonstrated the deployment of a static website on an AWS EC2 instance running Red Hat Enterprise Linux 10.2.

NGINX was installed and enabled as a system service. A basic HAWKEYE HTML page was created, and the deployment was extended by downloading the Start Bootstrap Landing Page template, extracting its files, and copying the production distribution into the NGINX web root.

The experiment also demonstrated SSH access, Linux file permissions, shell redirection, command-line file management, and SELinux context restoration.

The final landing page was accessible through the instance's public DNS over HTTP, completing the objective of hosting a static website on EC2.